

BÀI 8- ÔN TẬP & GIẢI ĐÁP CÁC CÂU HỎI ÔN TẬP

Câu 1

- 1.1. Giải thích ý nghĩa và cách sử dụng , cho ví dụ minh họa về Toán tử lấy địa chỉ của biến &, biến tham chiếu và cách tạo biến tham chiếu, dấu hoa thị *
- 1.2. Hãy trình bày về biến toàn cục , biến cục bộ, biến tĩnh static
- 1.3- a) Nêu định nghĩa, cách khai báo Con trỏ.
b) Giải thích toán tử **new**, toán tử **delete**, cho ví dụ.
c) Hằng con trỏ NULL ,
d) Tại sao phải dùng con trỏ (con trỏ dùng để làm gì)?

Câu 2. Cho cấu trúc nút-phần tử hàng đợi liên kết: `struct Node { int du_lieu; Node *next; };`
và cấu trúc của hàng đợi liên kết : `struct Queue { Node *front, *rear;};`

Theo các cấu trúc trên hãy viết các hàm:

- 2.1. Khởi tạo hàng đợi liên kết rỗng ban đầu
- 2.2. Tạo nút mới nhận dữ liệu
- 2.3. Thêm nút vào cuối hàng đợi liên kết
- 2.4. Lấy ra giá trị phần tử đầu hàng đợi liên kết
- 2.5. Xóa phần tử đầu hàng đợi liên kết

Câu 3. Cho khai báo cấu trúc kiểu nút: `struct Node { int du_lieu; Node* next; };`
và cấu trúc danh sách liên kết đơn : `struct List{Node* head; Node* tail; };`

Theo các cấu trúc đã cho hãy viết các hàm :

- 3.1. Khởi tạo danh sách liên kết rỗng ban đầu
- 3.2. Tạo nút mới nhận dữ liệu
- 3.3. Thêm nút vào sau nút k của dslk đơn
- 3.4. Xóa nút sau nút p của dslk đơn
- 3.5. Tìm nút theo giá trị dữ liệu

Câu 4. 4.1 Nêu định nghĩa độ phức tạp thuật toán o-nhỏ $o(f(n))$
4.2. Giải thích sự khác nhau giữa o-nhỏ $o(f(n))$ với O-lớn $O(f(n))$
4.3. Tính độ phức tạp của thuật toán nhân 2 ma trận $A_{mp} \times B_{pn}$ (ma trận A có m hàng, p cột, ma trận B có p hàng, n cột).

Câu 5. 5.1. Độ phức tạp của thuật toán là gì, giải thích và nêu định nghĩa
5.2. Tính độ phức tạp thuật toán dựa trên những thao tác cơ bản nào?
5.3. Hãy nêu các cấp độ phức tạp về thời gian chuẩn thực hiện các thuật toán
5.4. Hãy chứng minh quy tắc tổng đánh giá thời gian thực hiện thuật toán :
Nếu $T_1(n) = O(f_1(n))$ và $T_2(n) = O(f_2(n))$ thì $T_1(n) + T_2(n) = O(\max(f_1(n), f_2(n)))$
5.5. Đánh giá thời gian thực hiện các câu lệnh C++ như thế nào?.

Câu 6. Thuật toán đệ quy - 6.1. Trình bày khái niệm đệ quy
6.2. Viết hàm C++ tính đệ quy n!
6.3. Viết hàm C++ tính đệ quy dãy số fibonacci

6.4. Tính độ phức tạp $O(f(n))$ thuật toán đệ quy $n!$

6.5. Trong bài toán Tháp Hà Nội hãy chứng minh rằng với n đĩa thì số lần tối thiểu chuyển hết n đĩa từ cọc A sang 1 trong 2 cọc phụ B hay C là $2^n - 1$

Câu 7. 7.1. Trình bày cấu trúc dữ liệu kiểu ngăn xếp stack

7.2. Trình bày cài đặt stack bằng mảng 1 chiều

7.3. a) Trình bày cài đặt stack liên kết ,

b) các phép thêm phần tử vào đỉnh stack,

c) lấy ra giá trị ở đỉnh stack,

d) xóa phần tử đỉnh stack.

Câu 8. 8.1. Vẽ sơ đồ khối thuật toán sắp xếp nổi bọt dãy số $a[]$ có n phần tử theo chiều tăng dần giá trị

8.2. Theo sơ đồ khối thuật toán đã vẽ ở mục 8.1 hãy viết hàm sắp xếp nổi bọt dãy số $a[]$ có n phần tử theo chiều tăng dần giá trị.

Câu 9. 9.1. Trình bày về ma trận thưa,

9.2. Trình bày cách lưu trữ dữ liệu trong máy tính đối với ma trận thưa

9.3. Vẽ sơ đồ khối thuật toán hoán vị ma trận thưa có n hàng m cột có q phần tử khác 0.

9.4. viết chương trình C++ cho sơ đồ khối thuật toán đã vẽ ở mục 9.3

Câu 10. 10.1. Vẽ sơ đồ khối thuật toán sắp xếp chọn theo chiều tăng dần khi nhập các số vào từ bàn phím.

10.2. Viết chương trình C++ sắp xếp chọn theo chiều tăng dần khi nhập các số vào từ bàn phím theo sơ đồ khối thuật toán đã vẽ ở mục 10.1 ở trên

Câu 11. Sắp xếp chèn.

11.1. Vẽ sơ đồ khối thuật toán nhập các số từ bàn phím và so sánh sắp xếp chèn ngay các số mới nhập theo trật tự tăng dần giá trị (chú ý : mỗi lần nhập số mới phải so sánh với các số đã nhập để tìm vị trí chèn vào dãy số chứ không phải nhập cả dãy số vào rồi mới sắp xếp).

11.2. Viết chương trình theo sơ đồ khối thuật toán đã vẽ ở mục 11.1

11.3. Tính độ phức tạp của thuật toán trên theo độ đo $O(f(n))$.

Câu 12. Cho danh sách lớp có n người, có các cột mã sinh viên, họ tên, điểm trung bình học kỳ. Vẽ sơ đồ thuật toán và viết chương trình tìm và lập ra danh sách những người có điểm ≥ 8 , danh sách những người có điểm cao nhất lớp, danh sách mới này cũng có các cột mã sinh viên, họ tên, điểm trung bình học kỳ

Câu 13. 13.1. Trình bày thuật toán tìm kiếm theo chia đôi (tìm kiếm nhị phân) trong một tập đã được sắp thứ tự

13.2. Vẽ sơ đồ khối thuật toán tìm kiếm nhị phân đã trình bày ở mục 13.1

13.3. Viết hàm C++ cho sơ đồ thuật toán đã vẽ ở mục 13.2

Câu 14.

14.1. Vẽ sơ đồ khối thuật toán Euclid tìm ước số chung lớn nhất (USCLN) của 2 số nguyên a, b .

14.2. Viết hàm C++ cho sơ đồ khối thuật toán Euclid tìm ước số chung lớn nhất (USCLN) của 2 số nguyên a, b đã vẽ ở mục 14.1.

14.3. Hãy chứng minh rằng độ phức tạp của thuật toán tìm USCLN của Euclid có độ phức tạp là $O(\log_2 N)$

GIẢI ĐÁP

Câu 1

- 1.1. Giải thích ý nghĩa và cách sử dụng, cho ví dụ minh họa về Toán tử lấy địa chỉ của biến &, biến tham chiếu và cách tạo biến tham chiếu, dấu hoa thị *
- 1.2. Hãy trình bày về biến toàn cục, biến cục bộ, biến tĩnh static
- 1.3. a) Nêu định nghĩa, cách khai báo Con trỏ. b) Giải thích toán tử **new**, toán tử **delete**, cho ví dụ. c) Hằng con trỏ NULL, d) Con trỏ dùng để làm gì?

Trả lời Câu 1.

1.1. Giải thích ý nghĩa và cách sử dụng, cho ví dụ minh họa về Toán tử & lấy địa chỉ của biến, biến tham chiếu và cách tạo biến tham chiếu, dấu hoa thị *

a) Toán tử & dùng để lấy địa chỉ của một biến, cú pháp: &(a).

Ví dụ: cho khai báo biến `int a=6;` dùng lệnh `cout<<&(a)` sẽ cho địa chỉ ô nhớ chứa biến a đang có giá trị 6;

b) Biến tham chiếu và cách tạo biến tham chiếu

Biến tham chiếu là một biến được tạo ra từ một biến có trước, có cùng kiểu dữ liệu, cùng tham chiếu đến vùng nhớ như biến có trước đó. Để tạo ra một biến tham chiếu đối với biến **x** ta đặt tên biến là **x_ref**. Để khởi tạo biến **x_ref** ta đặt toán tử & ngay trước **x_ref**: `int &x_ref = x;`

Những xử lý làm thay đổi giá trị **x_ref** đều thay đổi giá trị **x**, địa chỉ của biến tham chiếu **x_ref** sẽ được xác định và không thay đổi khi bị thay đổi giá trị,

Ví dụ:

```
int x=9;      int &x_ref = x;
```

c) Dấu *

Dấu hoa thị * trong C++ được sử dụng với các chức năng:

- + dùng để khai báo biến con trỏ, ví dụ khai báo con trỏ p kiểu int ta viết `int *p;`
- + dùng ký hiệu phép nhân, ví dụ viết a nhân b là `a*b;`
- + dùng làm toán tử cho giá trị của địa chỉ ô nhớ chứa biến nào đó, ví dụ có biến a đang ở ô nhớ có địa chỉ &(a), khi đó `*(&(a))` sẽ cho giá trị của biến a.

1.2. Hãy trình bày về biến toàn cục, biến cục bộ, biến tĩnh static

3 loại biến:

- Biến toàn cục (**global variable**) được khai báo bên ngoài khối lệnh, bên ngoài hàm, có thể được truy xuất tại bất cứ dòng lệnh nào đặt bên dưới biến đó. Biến toàn cục tồn tại đến khi chương trình kết thúc.
- Biến cục bộ (**local variable**) được khai báo bên trong khối lệnh, bên trong hàm, có thể được truy xuất tại bất cứ dòng lệnh nào đặt bên dưới biến đó và trong cùng khối lệnh, bên trong hàm. Biến cục bộ bị hủy khi chương trình chạy ra ngoài khối lệnh, ra khỏi hàm chứa biến đó.
- Biến tĩnh được khai báo với từ khóa **static**, là loại biến lưỡng tính, vừa có tính chất của 1 biến toàn cục (global variables), vừa mang tính chất của 1 biến cục bộ (local variables).
Tính chất của biến toàn cục: biến không mất đi khi khối lệnh định nghĩa nó kết thúc, nó vẫn nằm trong vùng nhớ của chương trình và được tự động cập nhật khi khối lệnh đó được gọi lại. *Tính chất của biến cục bộ:* biến chỉ có thể được sử dụng trong khối lệnh, trong hàm mà nó được khai báo. Biến tĩnh static variables được tạo ra duy nhất một lần khi được gán giá trị lần đầu tiên và nó sẽ tiếp tục tồn tại trong suốt vòng đời của chương trình. Đây là sự khác biệt giữa biến tĩnh và biến cục bộ.

1.3.- a) Nêu định nghĩa, cách khai báo Con trỏ. b) Giải thích toán tử **new**, toán tử **delete**, cho ví dụ.

c) Hằng con trỏ NULL, d) Tại sao phải dùng con trỏ?

a) Một con trỏ (pointer) là một biến có lưu trữ địa chỉ của ô nhớ (gọi là phần liên kết) mà nó trỏ đến, một con trỏ thường có 2 thành phần là dữ liệu và liên kết, bản thân con trỏ có địa chỉ độc lập.

Kiểu dữ liệu của biến tại ô nhớ có địa chỉ mà con trỏ trỏ đến sẽ được coi là kiểu dữ liệu của con trỏ, dùng dấu * để khai báo con trỏ, lệnh khai báo con trỏ có dạng :

<tên kiểu dữ liệu> * <tên biến con trỏ>;

Ví dụ : int *p; // khai báo con trỏ p trỏ đến vùng nhớ kiểu int

b) toán tử **new**, toán tử **delete**, cho ví dụ.

new là thủ tục cấp phát bộ nhớ cho con trỏ, **delete** là thủ tục xóa con trỏ, thu hồi bộ nhớ của con trỏ ví dụ :

```
Node* node = new Node; node = list.head; list.head = node->next; delete node;
```

c) - Hằng con trỏ NULL là con trỏ không trỏ vào đâu cả

d) Tại sao phải dùng con trỏ ?

Trong tổ chức dữ liệu có 2 phương thức là tổ chức theo dữ liệu tĩnh và dữ liệu động. Dữ liệu động là dữ liệu mà được sinh ra khi cần đến và bị xóa bỏ khi không cần đến trong quá trình thực hiện chương trình. Để tổ chức theo dữ liệu động phải thông qua một biến con trỏ, trỏ tới bộ nhớ của biến động.

Câu 2. Cho Cấu trúc nút-phần tử hàng đợi liên kết

```
struct Node { int du_lieu; Node *next; };
```

Và Cấu trúc của hàng đợi liên kết : struct Queue { Node *front, *rear;};

Theo các cấu trúc đã cho hãy viết các hàm :

2.1. Khởi tạo hàng đợi liên kết rỗng ban đầu

2.2. Tạo nút mới nhận dữ liệu

2.3. Thêm nút vào cuối hàng đợi liên kết

2.4. Lấy ra (hiện) giá trị phần tử đầu hàng đợi

2.5. Xóa phần tử đầu hàng đợi

Trả lời

2.1. Khởi tạo hàng đợi liên kết rỗng ban đầu

```
void KhoitaoQ(Queue &q) {q.front = q.rear = NULL; }
```

2.2. Tạo nút mới nhận dữ liệu

```
Node *KhoitaoNode( int x )
{ Node *p = new Node; p->next = NULL; p->du_lieu = x; return p; }
```

2.3. Thêm phần tử vào cuối hàng đợi liên kết- hàm Push

```
void Push(Queue &q, Node *p) { if(!q.front) q.front = q.rear = p;
                               else { q.rear->next = p; q.rear = p; }
}
```

2.4. Lấy (hiện giá trị) phần tử đầu hàng đợi liên kết-

```
int Front(Queue q) { if(q.front) { return q.front->du_lieu; } else return 0; }
```

2.5. Xóa phần tử đầu hàng đợi liên kết

```
int Pop(Queue &q)
{ if(q.front) { Node *p = q.front; q.front = p->next; int a = p->du_lieu; delete p; } return 0; }
```

Câu 3. Cho khai báo cấu trúc kiểu nút, cấu trúc danh sách liên kết đơn (dsk đơn) :

```
struct Node { int du_lieu; Node* next; };
struct List { Node* head; Node* tail; };
```

Theo các cấu trúc đã cho hãy viết các hàm :

3.1. Khởi tạo danh sách liên kết rỗng ban đầu

3.2. Tạo nút mới nhận dữ liệu

3.3. Thêm nút vào sau nút k của dslk đơn

3.4. Xóa nút sau nút p của dslk đơn

3.5. Tìm nút theo giá trị dữ liệu

Trả lời

3.1. Khởi tạo danh sách liên kết rỗng ban đầu

```
void KhoiTaoList(List& list) { list.head=NULL; list.tail=NULL; }
```

3.2. Tạo nút mới nhận dữ liệu

```
Node* KhoitaoNode(int du_lieu0) {Node* node = new Node;  
    node->du_lieu = du_lieu0; node->next = NULL; return node; }
```

3.3. Thêm nút vào sau nút k của dslk đơn

```
void ThemNutSauK(List& list, Node* node Node* k)  
{ if(k != NULL) { node->next = k->next; k->next = node;  
    if(list.tail==k) list.tail = node; }  
    else { list.tail->next = node ; list.tail = node; }  
}
```

3.4. Xóa nút sau nút p của dslk đơn

```
int XoaNutSauP(List& list, Node* p, int& a)  
{ if(p != NULL) { Node* q = p->next;  
    if (q != NULL)  
        { p->next = q->next; a = q->du_lieu;  
        if (list.tail == q) list.tail =p; delete q; return 1; }  
    return 0; }  
    return 0; }
```

3.5. Tìm nút theo giá trị dữ liệu

```
Node* TimNutData(List& list, int a) { Node* node = list.head;  
    while (node != NULL && node->du_lieu != a) node = node->next;  
    if(node != NULL) return node;  
    return NULL; }
```

Câu 4.

4.1. Nêu định nghĩa độ phức tạp thuật toán o-nhỏ $o(f(n))$

$T(n)=o(f(n))$ nếu với mọi hằng số c, tồn tại hằng số $n_0 > 0$ sao cho $T(n) < cf(n)$ với mọi $n \geq n_0$.

Như vậy, **$o(.)$** là **cận trên** của thời gian thực hiện thuật toán.

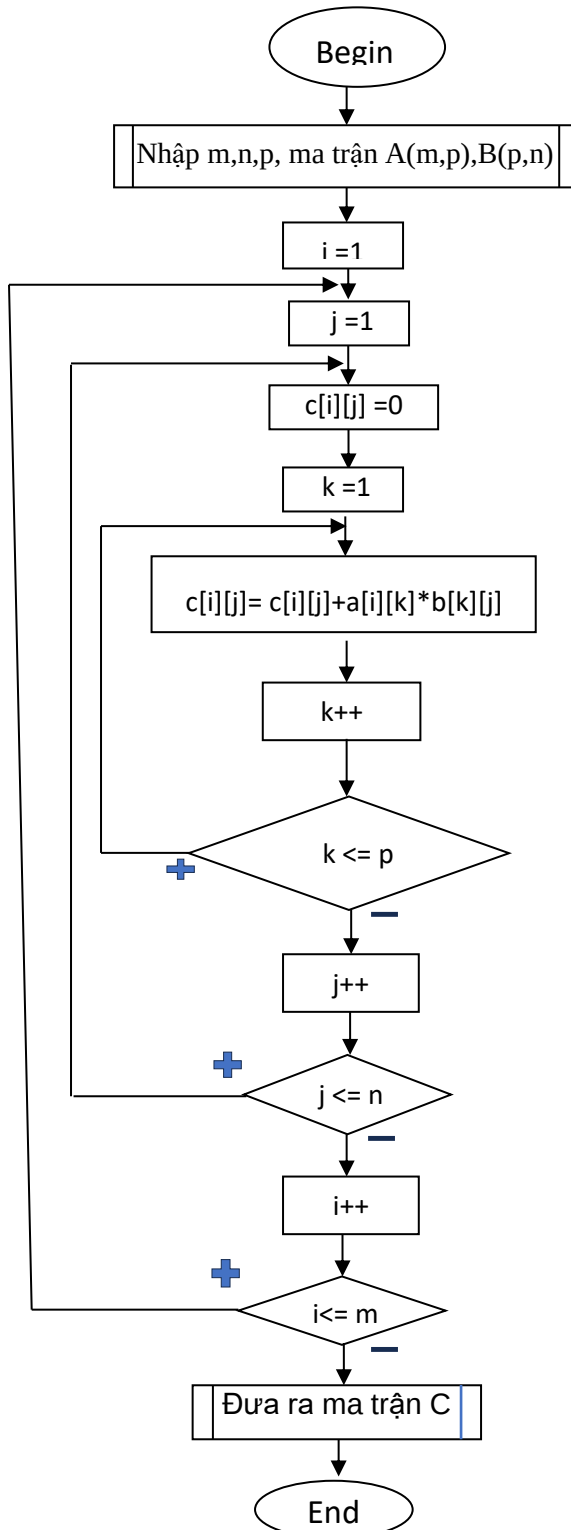
Ngoài ra, còn định nghĩa khác cho $o(.)$: **$T(n)=o(f(n))$** nếu $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = 0$

4.2. Giải thích sự khác nhau giữa o-nhỏ $o(f(n))$ với O-lớn $O(f(n))$

Trong định nghĩa **độ phức tạp O_lớn** là độ lớn của $T(n)$ được xác định như sau : giả sử n là số nguyên dương, $T(n)$ và $f(n)$ là các hàm thực không âm, khi đó $T(n)=O(f(n))$ nếu và chỉ nếu tồn tại các hằng dương c và n_0 sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$, như vậy hàm $f(n)$ là **cận trên** của $T(n)$.

Sự khác nhau giữa O_lớn và o_nhỏ là trong O_lớn $T(n)$ có thể $= f(n)$ còn trong o_nhỏ thì $T(n) < f(n)$.

4.3. Vẽ sơ đồ khối thuật toán nhân 2 ma trận $A_{m \times p} \times B_{p \times n}$ (ma trận A có m hàng, p cột, ma trận B có p hàng, n cột) và tính độ phức tạp O_lớn theo sơ đồ thuật toán đã vẽ.



Tính độ phức tạp thuật toán nhân 2 ma trận:

a) Phần nhập dữ liệu :

nhập m,n,p cần 3 thao tác cơ bản

nhập ma trận A có m hàng p cột có $m \times p$ thao tác cơ bản $\leq k^2$ ($k = \max(m,p)$)

nhập ma trận B có p hàng n cột có $p \times n$ thao tác cơ bản $\leq k^2$ ($k = \max(p,n)$)

b) Phần tính tích $C = A \times B$

Mỗi phần tử $c[i][j]$ cần $2p$ thao tác cơ bản – (2 phép + và * : $c[i][j] + a[i][k] \times b[k][j]$)

Cả ma trận C cần $m \times n \times 2p \leq 2k^3$ ($k = \max(m,n,p)$)

Kết luận, độ phức tạp thuật toán nhân 2 ma trận là $O(n^3)$

Câu 5.

5.1. Hãy trình bày về độ phức tạp của thuật toán

Độ phức tạp của thuật toán liên quan đến thời gian tính toán mà yếu tố chính là lượng thông tin đầu vào. Ký hiệu T là thời gian thực hiện thuật toán, khi đó $T(n)$ là một hàm số phụ thuộc vào số lượng đầu vào $n : T(n)$.

Khái niệm **độ phức tạp của thuật toán** được hiểu theo ý nghĩa là độ lớn của $T(n)$, được ký hiệu bằng chữ O (đọc là ô lớn) như sau : giả sử n là số nguyên dương, $T(n)$ và $f(n)$ là các hàm thực không âm, khi đó $T(n)=O(f(n))$ nếu và chỉ nếu tồn tại các hằng dương c và n_0 sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$. Như vậy, hàm $f(n)$ là **cận trên** của $T(n)$.

5.2. Để đánh giá thời gian thực hiện một thuật toán, phải làm như thế nào?

Để đánh giá thời gian thực hiện một thuật toán, phải đếm được số thao tác cơ bản mà thuật toán đó thực hiện, mỗi thao tác cơ bản thực hiện bao nhiêu lần, trong mỗi lần thực hiện bao nhiêu phép toán...

Những thao tác cơ bản có thể là phép đọc nhập dữ liệu; hiện dữ liệu; phép gán; cộng, trừ, nhân, chia, so sánh ... Thời gian thực hiện một thao tác cơ bản được tính là 1 đơn vị thời gian,

Nếu một thao tác gồm nhiều thao tác cơ bản thì thời gian thực hiện nó là tổng số lượng các thao tác cơ bản;

Ví dụ : Trong thao tác **if(điều kiện) S1 else S2**; thời gian thực hiện là cận trên của thời gian thực hiện lệnh **S1, S2**. Với thời gian để thực hiện **S1** là t_1 , thời gian để thực hiện **S2** là t_2 thì thời gian để thực hiện **if(điều kiện) S1 else S2**; là $\max(t_1, t_2)$ - là cận trên của (t_1, t_2) . Cụ thể : nếu $t_1 = 8, t_2 = 10$ thì $\max(t_1, t_2) = 10$

5.3. Hãy nêu các cấp độ phức tạp về thời gian chuẩn thực hiện các thuật toán

Trong phân tích thuật toán, người ta đã đưa ra một số cấp thời gian chuẩn thực hiện thuật toán như sau :

$O(1)$ - Độ phức tạp hằng số, khi số phép tính/thời gian chạy chương trình/dung lượng bộ nhớ không phụ thuộc đầu vào n , ví dụ : các thao tác hệ thống, hay mỗi phép $+, -, *, /, \%, \&\&, ||, !$

$O(\log(n))$ - Độ phức tạp logarit. Ví dụ thuật toán Euclid tìm USCLN, với $n = 8$ thì $\log(8)=3$.

$O(n)$ - độ phức tạp tuyến tính $O(n)$ - Độ phức tạp tuyến tính, khi số phép tính/thời gian chạy chương trình/dung lượng bộ nhớ phụ thuộc đầu vào n .

$O(n \log n)$ - Độ phức tạp $n \log n$.

$O(P(n))$ - - độ phức tạp đa thức, P là đa thức bậc từ 2 trở lên. Ví dụ bài toán tính định thức ma trận.

$O(n^2)$ - Độ phức tạp bình phương

$O(2^n)$ - Độ phức tạp số mũ (2^n)

Các cấp độ này cho độ đo để chọn thuật toán nào tốt hơn khi giải quyết 1 bài toán..

5.4. Hãy chứng minh quy tắc tổng đánh giá thời gian thực hiện thuật toán :

Nếu $T_1(n) = O(f_1(n))$ và $T_2(n) = O(f_2(n))$ thì $T_1(n) + T_2(n) = O(\max(f_1(n), f_2(n)))$

Chứng minh : theo định nghĩa $O()$:

$T_1(n) = O(f_1(n))$ nên $\exists c_1, n_1$ sao cho $T_1(n) \leq c_1 f_1(n)$ với $\forall n \geq n_1, (\exists$ đọc là *tồn tại*, $\forall$ - với mọi)

$T_2(n) = O(f_2(n))$ nên $\exists c_2, n_2$ sao cho $T_2(n) \leq c_2 f_2(n)$ với $\forall n \geq n_2$,

Đặt $n_0 = \max(n_1, n_2)$, khi đó với $\forall n \geq n_0$ ta có :

$$\begin{aligned} T_1(n) + T_2(n) &\leq (c_1 + c_2) \max(f_1(n), f_2(n)) = c \max(f_1(n), f_2(n)) \text{ với } c = c_1 + c_2 \\ &= O(\max(f_1(n), f_2(n))) \end{aligned}$$

Tổng quát : khi một bài toán được chia thành nhiều bài toán con thì **Độ phức tạp** thuật toán của bài toán sẽ lấy theo cấp **Độ phức tạp lớn nhất** trong các cấp độ thành phần.

5.5.Đánh giá thời gian thực hiện các câu lệnh C++ như thế nào?

Để tính độ phức tạp thực hiện một chương trình, phải biết được từng câu lệnh thực hiện bao nhiêu lần. Tùy theo loại lệnh sẽ có các chỉ tiêu đánh giá như sau:

1.Lệnh gán(=), lệnh nhập dữ liệu (cin>>) , lệnh hiện dữ liệu(cout<<) có thời gian thực hiện là $O(1)$;

2.Lệnh phức hợp để trong cặp {...} được đánh giá theo quy tắc tổng độ phức tạp ;

3.Lệnh if <điều kiện> s1 else s2 ;

có độ phức tạp là $O(\max(f_1(n), f_2(n)))$, trong đó $f_1(n), f_2(n)$ là cận trên của độ phức tạp S1, S2.

4.Lệnh switch case được đánh giá như lệnh if, số lượng case được tính nhân với số vòng lặp ngoài.

5.Lệnh lặp for, while, do...while, while : nếu độ phức tạp thực hiện trong thân vòng lặp là $f(n)$ và số lần lặp tối đa là $g(n)$ thì độ phức tạp thực hiện một lệnh lặp là $O(f(n)*g(n))$.

6.Đối với hàm đệ quy.

Xét hàm đệ quy tính giai thừa sau.

```
int giai thua(int n)
{ if (n <= 1)
  return 1;
  return n*giai thua(n-1);
}
```

Ta đặt Độ phức tạp thực hiện hàm với n là $T(n)$. Với $n=1$ thì Độ phức tạp thực hiện hàm là $O(1)$. Với $n > 1$, Độ phức tạp thực hiện hàm chính là thời gian thực hiện lệnh **gán $giai thua = n*giai thua(n-1)$ và $return\ giai thua$** , tức là bằng $O(1)$ cộng với $T(n-1)$.

Ta có quan hệ đệ quy :

$T(1) = O(1)$; // $O(1)$ là độ phức tạp hằng số

$T(n) = O(1) + T(n-1)$

$T(n-1) = O(1) + T(n-2)$

....

$T(2) = O(1) + T(1)$

Ta bắt đầu từ $T(n)$ cho đến $T(n-1), T(n-2), \dots, T(2), T(1)$:

$T(n) = O(1) + O(1) + \dots + O(1) + T(1) = (n-1)O(1) + T(1) = n*O(1) = nc_1 = O(n)$. - Độ phức tạp tuyến tính.

Câu 6. Thuật toán đệ quy 6.1. Khái niệm đệ quy

Đệ quy là hiện tượng một hàm tham trỏ đến chính nó.

Một hàm đệ quy gồm hai phần:

1. **Phần neo** (anchor – cỏi neo dựng neo thuyền, tàu bố) trong đó tác động của hàm được đặc tả cho một hay nhiều tham số.
2. **Phần đệ quy**, trong đó tác động cần được thực hiện cho giá trị hiện thời của các tham số được định nghĩa bằng các tác động hay giá trị được định nghĩa trước đó.

Ví dụ 1: để tính giai thừa có thể mô tả đệ quy :

ở đây phần **neo** là $0! = 1$

đệ quy là với $n > 0$ thì $n! = n*(n-1)!$

Kết quả được tính ngược và cho giá trị cuối cùng

$n! = n*(n-1)!$

$(n-1)! = (n-1)*(n-2)!$

$(n-2)! = (n-2)*(n-3)!$

.....

$3! = 3*(3-1)!$

$2! = 2*(2-1)! = 2*1!$

$1! = 1*(1-1)! = 1*0!$

$0! = 1$ cuối cùng đi đến phần neo.

6.2.Hàm đệ quy n!

```
int giaithua(int n)
{ if(n==0)
  { return 1 ;}
  return n*giaithua(n-1);
}
```

6.3.Hàm đệ quy fibonacy

Hàm fibonaci

```
1
1
2
3    với n> 2 mỗi số sau là tổng của hai số trước nó
5
8    Neo {Fib(1)=1
          {Fib(2)=1
          Độ quy: với n>2, Fib(n) = Fib(n-1)+Fib(n -2)
int Fib(int n)
{ if(n<=2)
  {return 1;}
  return (Fib(n-1)+ Fib(n-2));
}
```

6.4.Độ phức tạp thuật toán đệ quy n!

Xét hàm đệ quy tính giai thừa sau.

```
int giaithua(int n)
{ if (n <= 1)
  return 1;
  return n*giaithua(n-1);
}
```

Ta đặt Độ phức tạp thực hiện hàm với n là T(n). Với n=1 thì Độ phức tạp thực hiện hàm là O(1). Với n >1 , Độ phức tạp thực hiện hàm chính là thời gian thực hiện lệnh *gán giaithua = n*giaithua(n-1) và return giaithua*, tức là bằng O(1) cộng với T(n-1) .

Ta có quan hệ đệ quy :

$T(1) = O(1); // O(1)$ là độ phức tạp hằng số

$T(n) = O(1) + T(n-1)$

$T(n-1)=O(1)+T(n-2)$

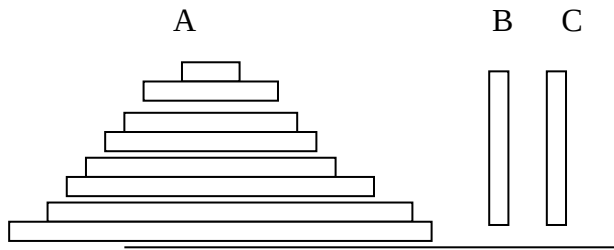
....

$T(2)=O(1)+T(1)$

Ta bắt đầu từ T(n) cho đến T(n-1),T(n-2),...T(2),T(1) :

$T(n) = O(1)+ O(1)+...+O(1)+T(1) = (n-1)O(1)+T(1) = n*O(1)= nc_1 = O(n)$. - Độ phức tạp tuyến tính.

6.5. Bài toán Tháp Hà Nội



Cần phải chuyển các đĩa kích thước khác nhau từ cọc bên trái sang cọc bên phải, với các luật sau:

1. Khi di chuyển một đĩa, nó phải được đặt vào một trong ba cọc đã cho
2. Mỗi lần chỉ di chuyển một đĩa và phải là đĩa ở trên cùng
3. Đĩa lớn hơn không được phép nằm trên đĩa nhỏ hơn. Cho N là số đĩa.

Ta làm đệ quy như sau :

Với $n=1$, việc chuyển đĩa từ $A \rightarrow C$ là giải được.

- 1, Giả sử đã chuyển được đối với $n-1$ đĩa trên cùng ở $A \rightarrow B$
2. Chuyển đĩa thứ $n \rightarrow$ cột C (C là cọc phụ)
3. Chuyển $n-1$ đĩa từ $B \rightarrow C$, dùng A là cọc phụ.

Chương trình :

```
#include <iostream>
using namespace std;
```

```
void Move(int n, char Dau, char Phu, char Cuoi)
{ if(n==1)cout<<"    Chuyen dia tu coc "<<Dau<<" sang coc "<<Cuoi;
  else
  { Move(n-1,Dau,Cuoi,Phu);cout<<"\n";
    Move(1,Dau,' ',Cuoi);cout<<"\n";
    Move(n-1,Phu,Dau,Cuoi);cout<<"\n";  }
}
```

```
int main()
{ cout << "\n\n    THAP HA NOI \n\n" << endl;
  const char coc1 ='A',coc2 ='B',coc3 ='C';
  int sodia;// số đĩa
  cout<<"  Nhập số đĩa : ";cin>>sodia;
  Move(sodia,coc1,coc2,coc3);
  system("pause");
  return 0;
}
```

Trong bài toán Tháp Hà Nội hãy chứng minh rằng với n đĩa thì số lần tối thiểu chuyển hết n đĩa từ cọc A sang 1 trong 2 cọc phụ B hay C là $2^{n-1}-1$

Ta làm đệ quy như sau :

Với $n=1$, việc chuyển đĩa từ $A \rightarrow C$ là giải được.

- 1, Giả sử đã chuyển được đối với $n-1$ đĩa trên cùng ở $A \rightarrow B$
2. Chuyển đĩa thứ $n \rightarrow$ cột C (C là cọc phụ)

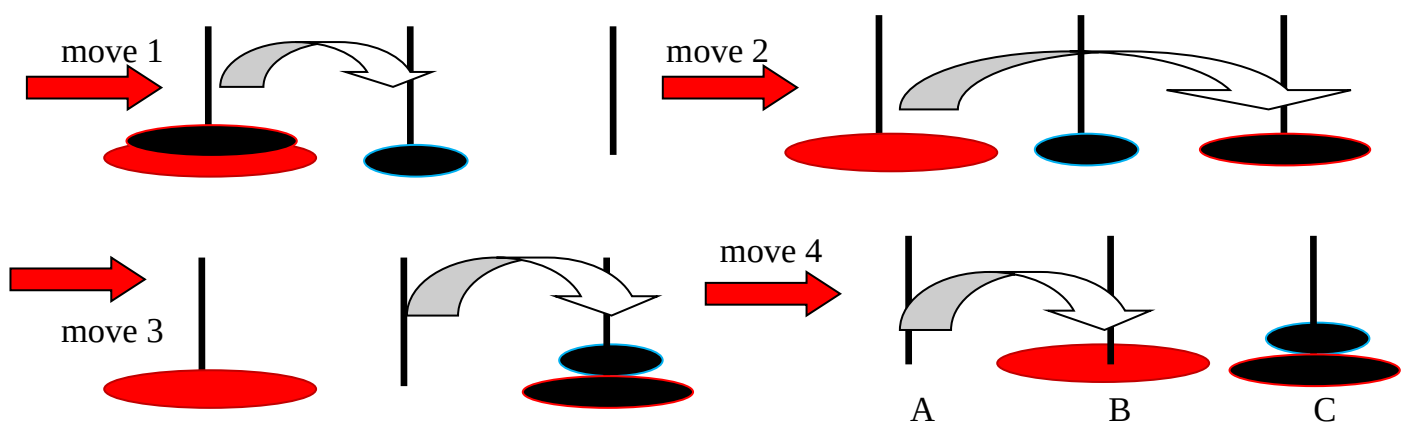
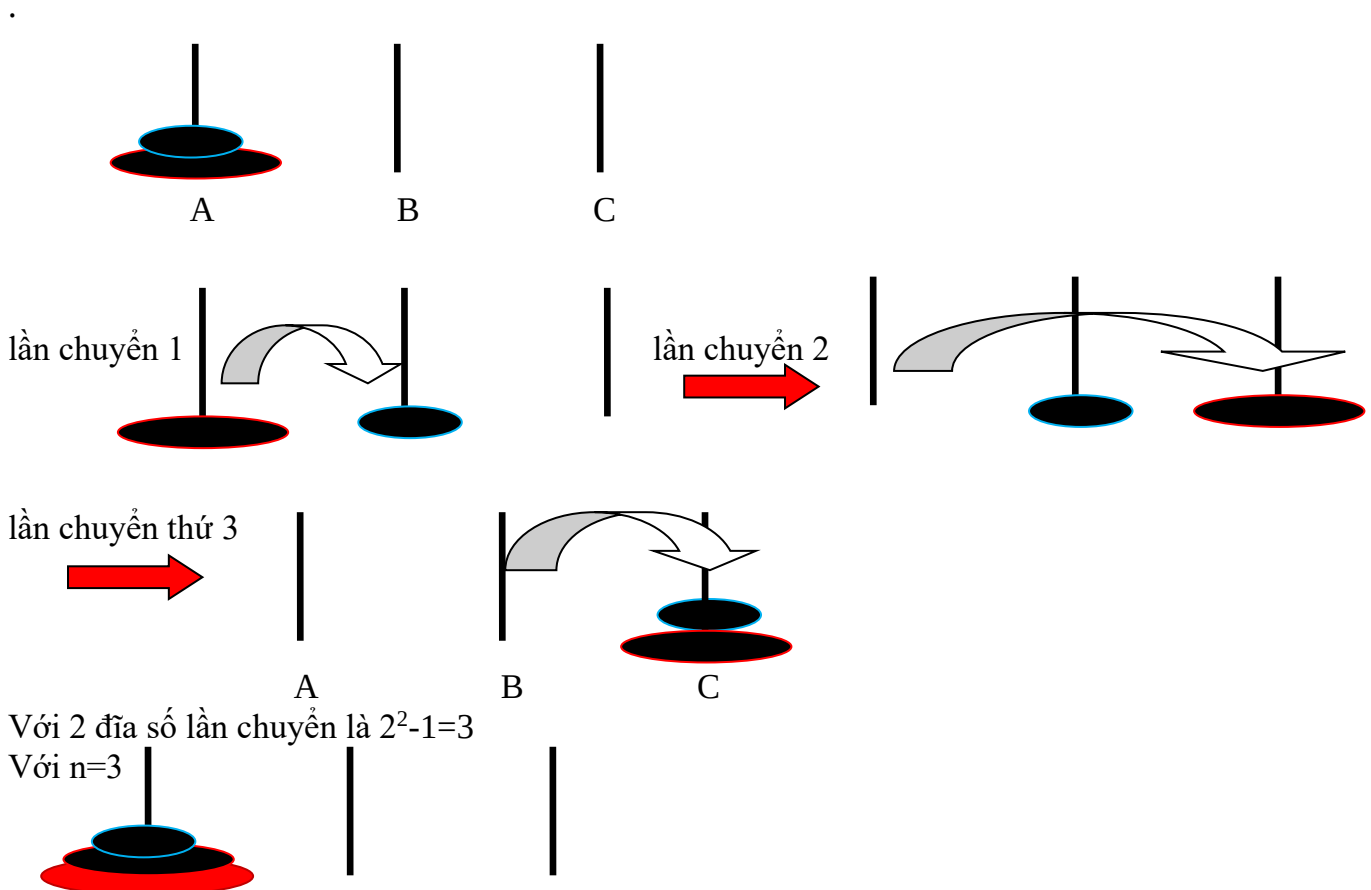
3. Chuyển $n-1$ đĩa từ B $\rightarrow$ C, dùng A là cọc phụ.

Ta cần chứng minh mệnh đề : bài toán tháp Hà nội chuyển n đĩa từ cọc A sang C có số lần chuyển là $2^n - 1$.

Chứng minh theo quy nạp.

Với $n=1$, việc chuyển đĩa từ A $\rightarrow$ C số lần chuyển là 1

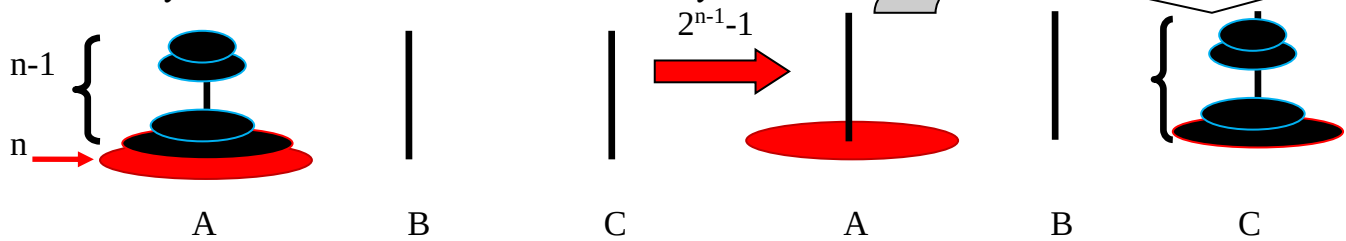
Với $n=2$, việc chuyển đĩa từ A $\rightarrow$ C là :



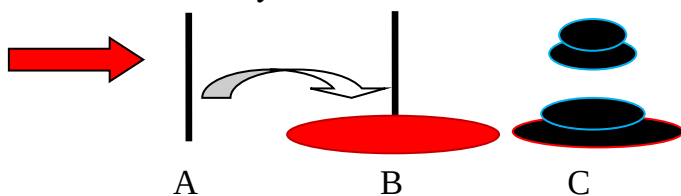
Từ cột C, chuyển 2 đĩa về B thì như đã xét sẽ cần 3 lần chuyển, vậy tổng số lần chuyển 3 đĩa là $7 = 2^3 - 1$

...
Giả thiết số lần chuyển $(n-1)$ đĩa là $2^{n-1} - 1$, ta chứng minh công thức này đúng với $n : 2^n - 1$.

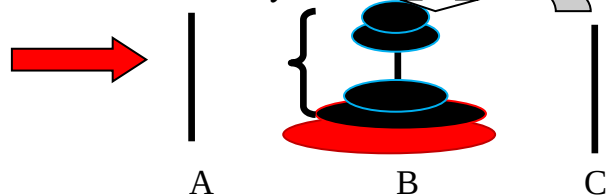
Phần 1 chuyển n-1 đĩa từ A->C cần $2^{n-1}-1$ lần chuyển



Tiếp theo chuyển đĩa thứ n sang cột B.
1 lần chuyển



Phần cuối cùng chuyển (n-1) đĩa từ C ->B
cần $2^{n-1}-1$ lần chuyển



Tổng số lần chuyển là $(2^{n-1}-1)+1+(2^{n-1}-1)=2*(2^{n-1}-1)+1=2^n-1$

Câu 7

7.1. Trình bày cấu trúc dữ liệu kiểu ngăn xếp stack

7.2. Cài đặt stack bằng mảng 1 chiều

7.3. Cài đặt stack liên kết, các phép thêm phần tử vào đỉnh stack, lấy ra giá trị ở đỉnh stack, xóa phần tử đỉnh stack.

Trả lời

7.1) Ngăn xếp- stack là một cấu trúc dữ liệu trừu tượng, xử lý theo kiểu vào sau ra trước (last in first out – LIFO). Có thể cài đặt stack theo kiểu một mảng(array) 1 tuyến tính hay danh sách liên kết đơn, trong đó các thao tác xử lý đều thực hiện ở một đầu gọi là đỉnh ngăn xếp.

7.2) Cài đặt stack kiểu mảng 1 chiều

Khai báo mảng 1 chiều `int stack[]` lưu trữ stack, biến `int top` lưu chỉ số của phần tử ở đỉnh `stack[top]` và được khởi gán `top = -1`, một biến `sizestack` lưu kích thước của stack.

Muốn thêm phần tử vào đỉnh stack thì cần kiểm tra stack đầy chưa (`top < sizestack`), nếu chưa đầy thì tăng `top++` rồi mới đưa dữ liệu vào đỉnh stack bằng lệnh `stack[top] = <dữ liệu mới>;`

Muốn lấy ra giá trị ở đỉnh ngăn xếp thì dùng lệnh gán giá trị ở đỉnh stack cho biến xuất ra: `<biến xuất giá trị> = stack[top];`

Muốn xóa đỉnh stack chỉ cần giảm `top--`, đỉnh mới sẽ là `stack[top];`

7.3) Cài đặt ngăn xếp liên kết (Nxlk) bằng danh sách liên kết đơn (Dslkđơn)

Ngăn xếp liên kết là một trường hợp riêng của dslkđơn khi mà mọi thao tác xử lý đều thực hiện ở đầu danh sách, vì vậy để cài đặt Nxlk phải sử dụng các khai báo các kiểu cấu trúc của Dslkđơn.

Khai báo kiểu cấu trúc nút của Nxlk: `struct Node { int du_lieu; Node* next; };`

Khai báo cấu trúc Nxlk: `struct LinkedStack { Node* head; };`

Khởi tạo Nxlk: `KhởiTaoStack(LinkedStack& s) { s.head=NULL; }`

Khởi tạo nút mới để nhận dữ liệu cho Nxlk:

```
Node* KhoitaoNode(int du_lieu0)
```

```
{ Node* p = new Node; p->du_lieu = du_lieu0; p->next = NULL; return p; }
```

- Khai báo hàm isEmpty - kiểm tra ngăn xếp liên kết có rỗng không

```
bool isEmpty(LinkedStack s) { return (s.head == NULL); }
```

- Thêm phần tử vào đỉnh ngăn xếp liên kết

- Kiểm tra, nếu ngăn xếp liên kết rỗng thì phần tử mới hiện thời là đầu ngăn xếp liên kết, ở đỉnh ngăn xếp; nếu ngăn xếp không rỗng thì phải đẩy nút đang ở đỉnh xuống rồi đưa nút mới hiện thời vào đỉnh ngăn xếp liên kết.

```
void ThemvaoDinh(LinkedStack &s, int du_lieu0)
```

```
{ Node* pht = KhoitaoNode(int du_lieu0);
```

```
if (isEmpty(s)) { s.head = pht; }
```

```
else { pht->next = s.head; s.head = pht; }
```

```
}
```

Lấy và Xóa nút đỉnh Nxlk :

Kiểm tra xem ngăn xếp liên kết có rỗng hay không, nếu rỗng thoát ra, nếu không rỗng thì lưu lại nút đỉnh, gán head bằng next của nút đỉnh rồi xóa nút đỉnh;

```
int XoaDinh(LinkedStack &s)
```

```
{ if (!isEmpty(s)) { Node *x = s.head; int a = x->du_lieu; s.head = x->next;
```

```
delete(x); return a; }
```

```
else { cout << "Stack rỗng!" << endl; return 0; }
```

```
}
```

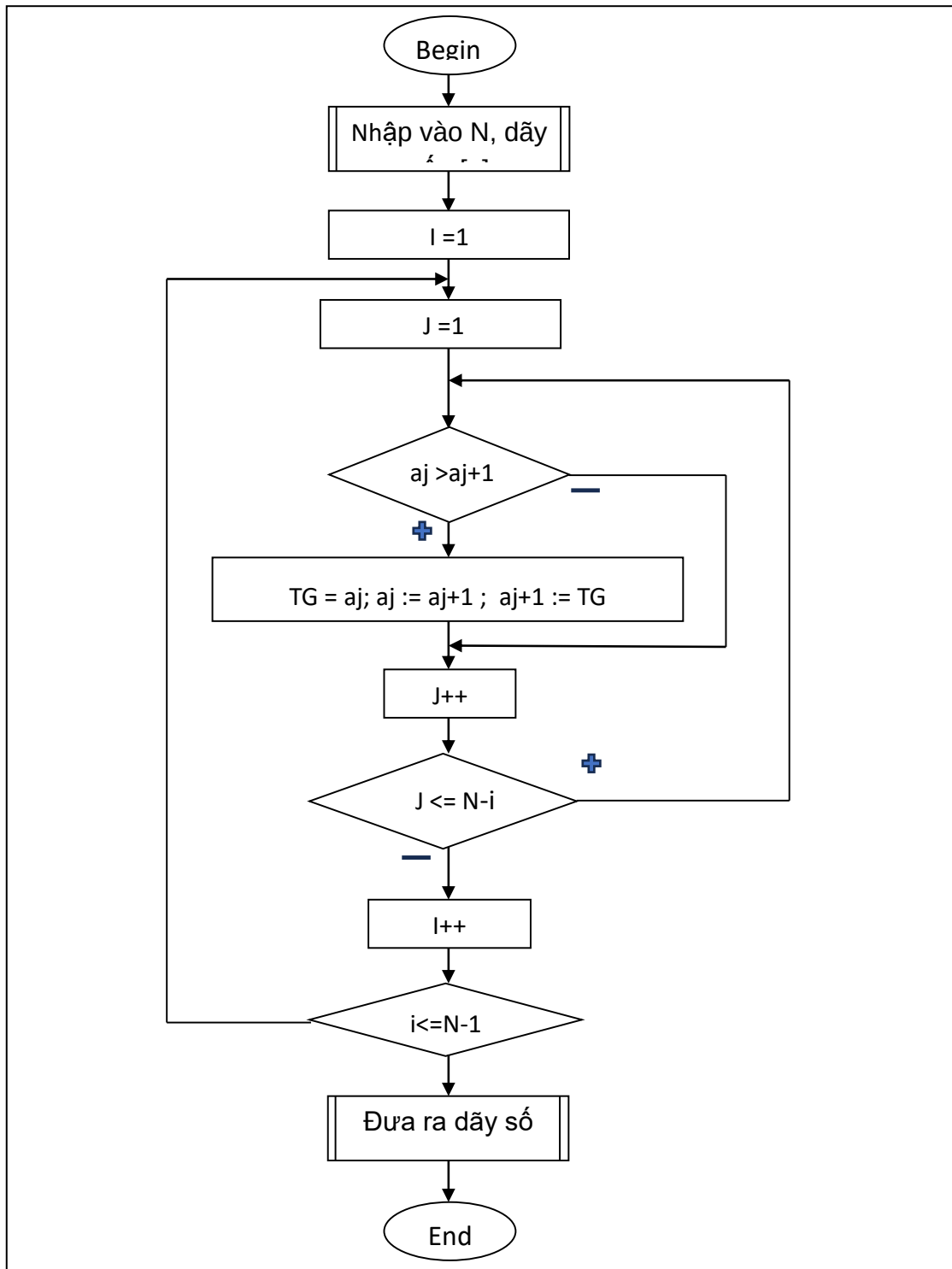
- Hiện dữ liệu của đỉnh

```
int Dinh(LinkedStack s)
```

```
{ if (!isEmpty(s)) { return s.head->du_lieu; } else { cout << "Stack rỗng!" ; return 0; }}
```

Câu 8.

8.1..Vẽ sơ đồ khối thuật toán sắp xếp nổi bọt dãy số $a[]$ có n phần tử theo chiều tăng dần giá trị



8.2.Theo sơ đồ khối đã vẽ ở mục 8.1 hãy viết hàm sắp xếp nổi bọt dãy số $a[]$ có n phần tử theo chiều tăng dần giá trị.

```
int main()
{ cout << "\n SAP XEP NOI BOT " << endl;
  int i,n,j;
  int tg,a[20];
  cout << "\n Nhap n: ";cin>>n;
```

```

for(i=0;i<n;i++){cout << "\n Nhập a["<i<<"]";cin>>a[i];}
for(i=1;i<=n-1;i++){//cout<<" i= "<i<<"\n";
for(j=0;j<n-i;j++){if(a[j]>a[j+1])swap(a[j],a[j+1]); } }
cout<< "\n DAY KET QUA SAP XEP \n" << endl;
for(i=0;i<n;i++){cout <<a[i]<<"-";}
return 0;
}

```

Câu 9.

9.1.Trình bày về ma trận thưa, cách lưu trữ dữ liệu đối với ma trận thưa

9.2. Vẽ sơ đồ khối thuật toán và viết chương trình C++ hoán vị ma trận thưa có m hàng n cột có q phần tử khác 0.

Trả lời . 9.1.Trình bày về ma trận thưa, cách lưu trữ dữ liệu đối với ma trận thưa

Ma trận thưa là dạng ma trận có rất ít phần tử có giá trị $\neq 0$. Khi nhập và xử lý chỉ xét các phần tử $\neq 0$.

Ví dụ : cho ma trận thưa A có m hàng, n cột, có q phần tử có giá trị $\neq 0$. Khi đó sẽ tổ chức lưu trữ dữ liệu trong một mảng A_{luu} có 3 cột và (q+1) hàng , trong đó $A[1][1]$ lưu m, $A[1][2]$ lưu n, $A[1][3]$ lưu q; các phần tử tiếp theo ở cột $A[][1]$ lưu chỉ số i, $A[][2]$ lưu chỉ số j, $A[][3]$ lưu $a[i][j] \neq 0$

	$A[:,1]$	$A[:,2]$	$A[:,3]$
1	m	n	q
2	i	J	$a[i][j] \neq 0$
3			
..	...	...	...
q+1			

Ví dụ . Cho ma trận thưa A có 6 hàng 6 cột, có 8 phần tử khác 0 :

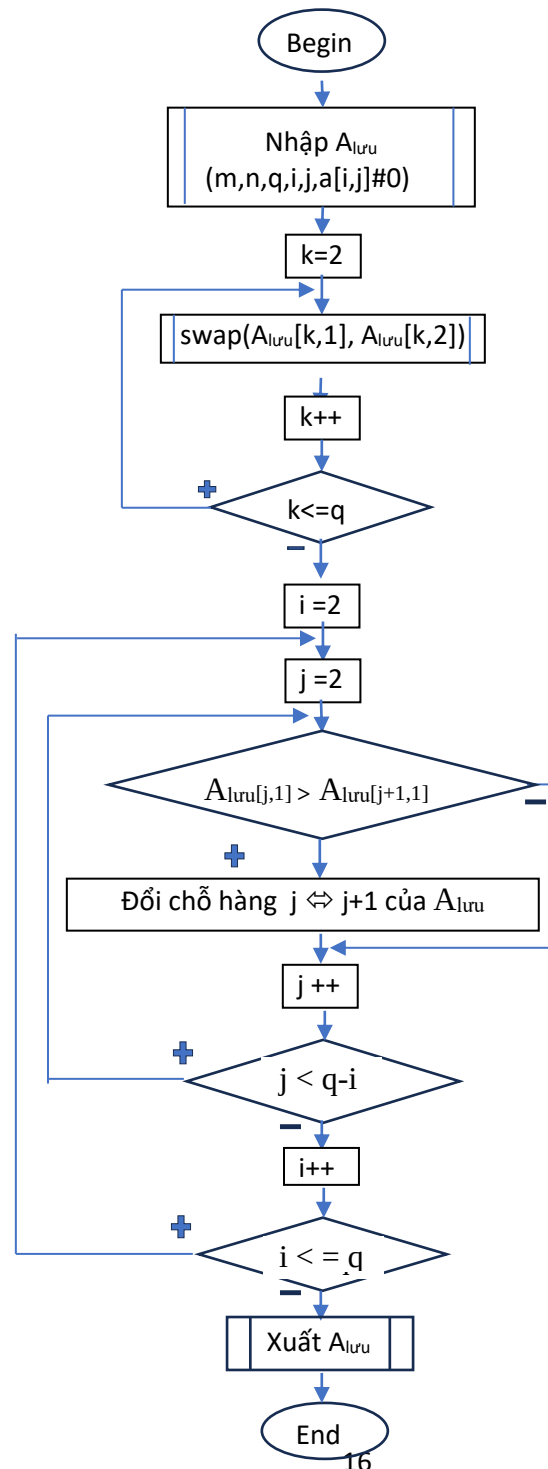
	1	2	3	4	5	6
1	15	0	0	22	0	-15
2	0	11	3	0	0	0
3	0	0	0	-6	0	0
4	0	0	0	0	0	0
5	91	0	0	0	0	0
6	0	0	28	0	0	0



Ta lưu trong ma trận A_{luu} có 9 hàng 3 cột :

A_{luu}	[0	6	6	8
	[1	1	1	15
	[2	1	4	22
	[3	1	6	-15
	[4	2	2	11
	[5	2	3	3
	[6	3	4	-6
	[7	5	1	91
	[8	6	3	28

9.2. Vẽ sơ đồ khối thuật toán và viết chương trình C++ hoán vị ma trận thưa có m hàng n cột có q phần tử khác 0. Xử lý trong ma trận A_{luu} : a) Đổi cột 1 $\leftrightarrow$ 2; b) Sắp xếp lại ma trận A_{luu} theo chiều tăng dần cột 1



Chương trình :


```

int main()
{
    cout << "    HOAN VI MA TRAN THUA !" << endl;
    int i,n=6,m=6,q=8;
    int a[10][10], Aluu[15][3];
    //cout<<"  Nhap so hang (n), so cot (m), q- so phan tu # 0 :";cin>>n>>m>>q;
    Aluu[1][1]=n;Aluu[1][2]=m; Aluu[1][3]=q;
    for(int k=2;k<=q+1;k++){cout<< "  Nhap i , j , a[i,j] # 0 : ";
cin>>Aluu[k][1]>>Aluu[k][2]>>Aluu[k][3];}
    cout<<"  Ma tran Aluu vua nhap : \n";
    for(int k=1;k<=q+1;k++){cout<<"  "<<Aluu[k][1]<<" - "<<Aluu[k][2]<<"-"<<Aluu[k][3]<<" \n";}
    cout<< "  swap column1 <=> column2 : \n";
    for(int k=1;k<=q+1;k++){swap(Aluu[k][1],Aluu[k][2]);}
    cout<<"  Ma tran Aluu sau swap column1 <=> column2 : \n";
    for(int k=1;k<=q+1;k++){cout<<"  "<<Aluu[k][1]<<" - "<<Aluu[k][2]<<"-"<<Aluu[k][3]<<" \n";}
    cout<< "  Sort by column1 : \n";
    for(i=1;i<=q;i++){//cout<<" i= "<<i<<"\n";
        for(int j=2;j<=q+1-i;j++){if(Aluu[j][1]>Aluu[j+1][1]){swap(Aluu[j][1],Aluu[j+1][1]);
swap(Aluu[j][2],Aluu[j+1][2]);swap(Aluu[j][3],Aluu[j+1][3]); } } }

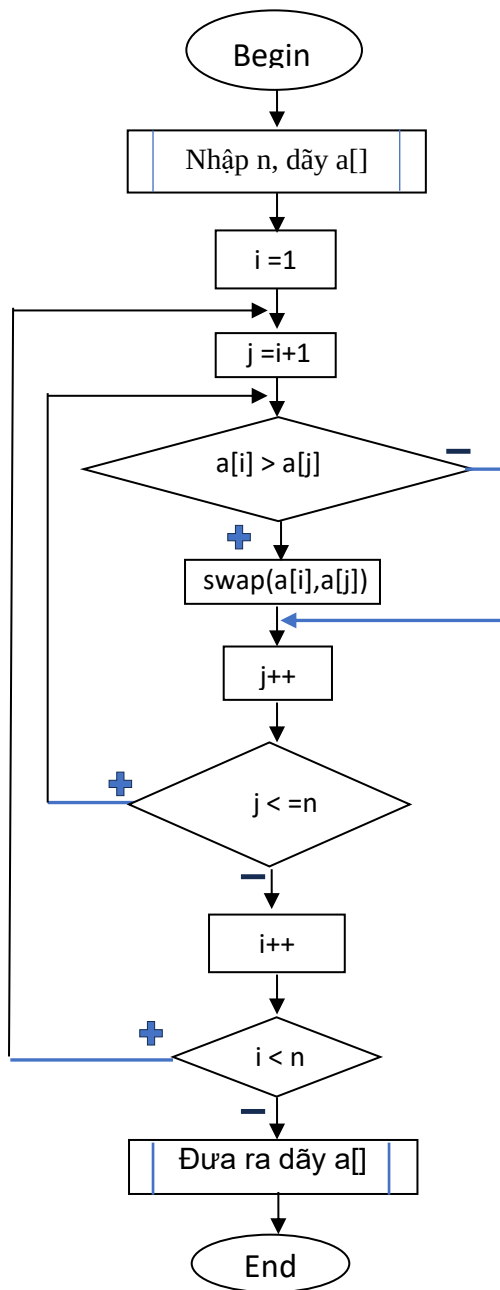
    cout<<"  Ket qua hoan vi Ma tran thua : \n";
    for(int k=1;k<=q+1;k++){cout<<"  "<<Aluu[k][1]<<" - "<<Aluu[k][2]<<"-"<<Aluu[k][3]<<" \n";}

    system("pause");
    return 0;
}

```

Câu 10.

10.1 – Vẽ sơ đồ khối thuật toán sắp xếp chọn theo chiều tăng dần khi nhập các số vào từ bàn phím.

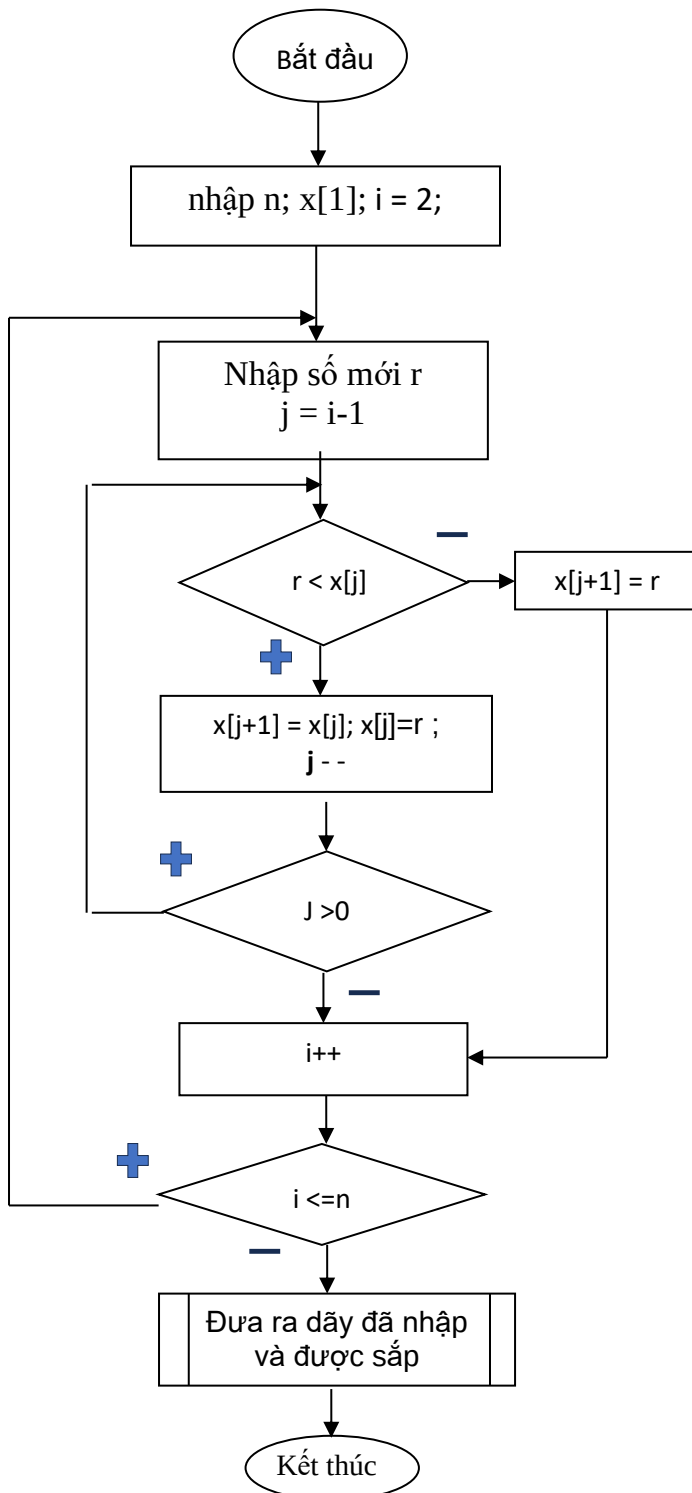


10.2. Viết chương trình C++ sắp xếp chọn theo chiều tăng dần khi nhập các số vào từ bàn phím theo sơ đồ khối thuật toán đã vẽ ở mục 10.1 ở trên

```

#include <iostream>
using namespace std;
int main()
{ cout << "\n SAP XEP CHON - SELECTION SORT " << endl;
  int i,n,j;
  int tg,a[20];
  cout << "\n Nhap n: ";cin>>n;
  for(i=1;i<=n;i++){cout << "\n Nhap a["<<i<<"]";cin>>a[i];}
  for(i=1;i<=n;i++)
    for(j=i+1;j<=n;j++){if(a[i]>a[j])swap(a[i],a[j]); }
  cout<< "\n DAY KET QUA SAP XEP \n" << endl;
  for(i=1;i<=n;i++){cout <<a[i]<<"-";}
  system("pause");
  return 0;
}
  
```

Câu 11. 11.1. Về sơ đồ khối thuật toán sắp xếp chèn ngay các số được nhập từ bàn phím sao cho các số đã nhập được đặt đúng theo trật tự tăng dần giá trị (chú ý : vừa nhập vừa sắp xếp chứ không phải nhập cả dãy số vào rồi mới sắp xếp).



11.2. Viết chương trình theo sơ đồ khối thuật toán đã vẽ ở mục 11.1, mỗi lần nhập thêm số mới sẽ được sắp xếp ngay và hiện luôn kết quả.

```

#include <iostream>
using namespace std;
int main()
{cout << " NHAP & SAP XEP CHEN DAY "
<< endl;
  int i,n,r;
  cout << " NHAP so phan tu n= ";cin>>n;
  int j,a[n];
  cout<<" Nhap a[1]= "; cin>>a[1];
  for(i=2;i<=n;i++){int k;
    cout<<" Nhap so moi r = "; cin>>r;
    j=i-1;
    while((j>0)&&(a[j]>r)){a[j+1]=a[j];j=j-1; }
    a[j+1]=r;k=i;
    cout << " DAY SO DA SAP XEP \n" ;
    for(int p=1;p<=k;p++) cout<<a[p]<<"-";
    system("pause()"); }
  return 0;
}
  
```

11.3(1đ) Tính độ phức tạp của thuật toán trên theo độ đo $O(f(n))$.

a) Phần nhập dữ liệu có tối đa $2n$ thao tác cơ bản

b) Phần so sánh sắp xếp có n lần lặp, trong mỗi lần lặp có tối đa so sánh n lần, độ phức tạp sẽ là n^2 .

c) Phần hiện kết quả sau mỗi lần nhập 1 số mới, có $(n-1)$ lần hiện, lần hiện nhiều nhất có n số, độ phức tạp là n^2

Tổng hợp lại độ phức tạp của cả thuật toán và chương trình là $O(n^2)$.

Câu 12. Cho danh sách lớp có n người, có các cột mã sinh viên, họ tên, điểm trung bình học kỳ. Vẽ sơ đồ thuật toán và viết chương trình tìm và lập ra danh sách những người có điểm ≥ 8 , danh sách những người có điểm cao nhất lớp, danh sách mới này cũng có các cột mã sinh viên, họ tên, điểm trung bình học kỳ.

Cách làm :

- Tạo một danh sách lớp - DS1 có n dòng, có các cột

Mã sinh viên	Họ tên	Điểm tb
--------------	--------	---------

- Tạo một danh sách DS2 - những người có điểm ≥ 8 có n dòng, có các cột

Mã sinh viên	Họ tên	Điểm tb
--------------	--------	---------

và một danh sách những người có điểm cao nhất lớp – DS3 cũng có n dòng, 2 cột :

Mã sinh viên	Họ tên
--------------	--------

- Nhập số phần tử n;
- Nhập danh sách DS1;
- Tạo DS2 : lần lượt kiểm tra so sánh điểm của từng sinh viên, ai có điểm ≥ 8 thì đưa Mã sinh viên.họ tên, điểm tb học kỳ của người đó sang DS2.
- Tìm điểm lớn nhất Max;
- Tạo DS3 : lần lượt kiểm tra so sánh điểm của từng sinh viên, ai có điểm bằng điểm Max thì đưa Mã sinh viên và họ tên người đó sang DS3.
- Hiện lại DS1, DS2, DS3 .

Hướng dẫn : dùng chỉ số i, j, k lần lượt làm chỉ số trong các danh sách DS1, DS2, DS3.

Khởi gán $i=1, j=0, k=0$.

Đặt tên các biến :

Trong DS1 : masv1[] – mã sinh viên; hoten1[] – họ tên ,tb1[] – điểm trung bình học kỳ,

trong DS2 : masv2[] – mã sinh viên; hoten2[] – họ tên ,tb2[] – điểm trung bình học kỳ,

Trong DS3 : masv3[] – mã sinh viên; hoten3[] – họ tên.

Max – điểm lớn nhất.

Phần tạo DS2 :

Cho $j=0$; Cho $i=1$ đến n thực hiện :

Nếu $(tb1[i] \geq 8)$ thì $\{j++; masv2[j]=masv1[i]; hoten2[j]=hoten1[i]; tb2[j]=tb1[i];\}$

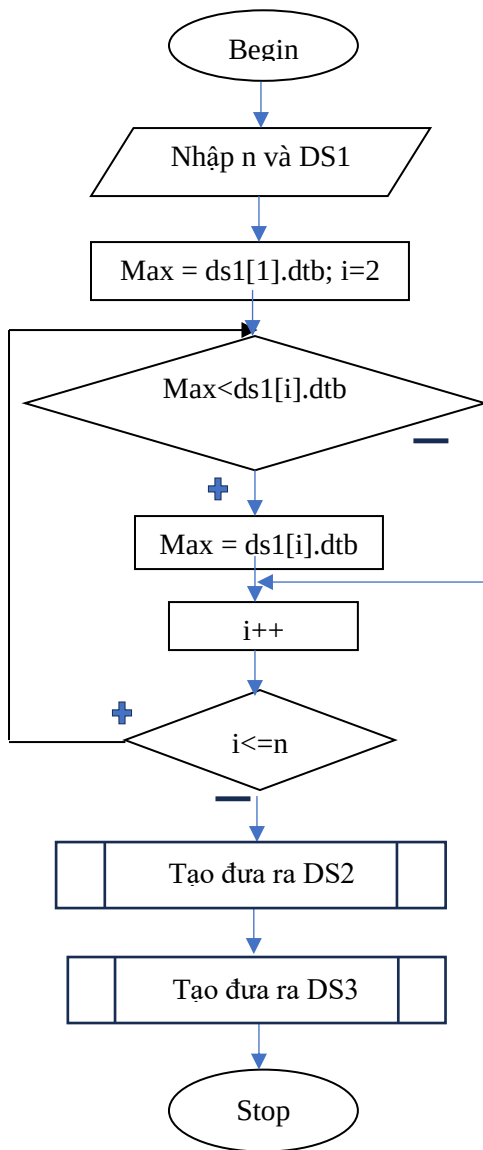
Phần tạo DS3 :

Phần tìm điểm Max :

$Max = tb1[1]$; Cho $i=2$ đến n thực hiện : nếu $(tb1[i] > Max)$ thì $Max=tb1[i]$;

Tạo DS3 : Cho $k=0$; Cho $i=1$ đến n thực hiện :

Nếu $(tb1[i] = Max)$ thì $\{k++; masv3[k]=masv1[i]; hoten3[k]=hoten1[i];\}$



12.2. Viết chương trình C++ cho sơ đồ thuật toán đã vẽ ở mục 12.1.

```

int main()
{
    int i,j,k,n;
    string masv1[50],hoten1[50], masv2[50],hoten2[50],
    masv3[50],hoten3[50];
    float tb1[50], tb2[50],tb3[50];
    cout<<"  Nhap N = ";cin>>n;
    for(i=1;i<=n;i++)
    {
        cin.ignore(); cout<<"  Nhap ma sv thu "<< i<< " : ";
        getline(cin,masv1[i]);
        cin.ignore(); cout<<" \n  Nhap Ho ten sv thu " <<i<<";
        getline(cin,hoten1[i]);
        cout<<" \n  Nhap diem trung binh hoc ky ";cin>>tb1[i];
    }
    j=0;
    for(i=1;i<=n;i++)
    if(tb1[i]>=8){j++; masv2[j]=masv1[i];hoten2[j]=hoten1[i];
    tb2[j]=tb1[i];}
    if(j>0){int q; cout<<" DANH SACH SV Co dtb >=8 : \n";
    for(q=1;q<=j;q++)cout<<" Ho ten : "<< hoten2[q]<<" |
    Diem tb : "<<tb2[q]<<" \n"; system("pause");}
    float Max = 0;
    for(i=1;i<=n;i++){if(tb1[i]>Max) Max=tb1[i];}
    k=0;
    for(i=1;i<=n;i++)
    { if(tb1[i]== Max)
    {k++;
    masv3[k]=masv1[i];hoten3[k]=hoten1[i];tb3[k]=tb1[i];} }
    if(k>0){cout<<" DANH SACH SV DAT MAX : \n";
    for(int p=1;p<=k;p++)cout<<" Ho ten: "<<hoten3[p]<<" | Masv :
    "<<masv3[p]<<" | Diem tb= "<<Max<<" \n"; }
    system("pause");
    return 0;
}
  
```

Câu 13. Tìm kiếm theo chia đôi (tìm kiếm nhị phân)

13.1. Trình bày thuật toán tìm kiếm theo chia đôi (tìm kiếm nhị phân) trong một tập đã được sắp thứ tự

Muốn tìm một phần tử trong một tập Δ đã được sắp thứ tự, lần thứ nhất ta chia đôi tập Δ thành A_1, A_2 ; Lúc đầu tìm phần tử trong A_1 , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi A_2 thành B_1, B_2 ; tiếp tục tìm phần tử trong B_1 , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi B_2 thành C_1, C_2 ... quá trình tiếp tục cho đến khi tìm được phần tử.

Theo cách chia đôi: tập A_1 có lực lượng = $1/2 \Delta \rightarrow 1/2^1$

B_1 có lực lượng = $1/4 \Delta \rightarrow 1/2^2$

C_1 có lực lượng = $1/8 \Delta \rightarrow 1/2^3$

Ở bước thứ n , tập được chia đôi có lực lượng $\rightarrow 1/2^n$

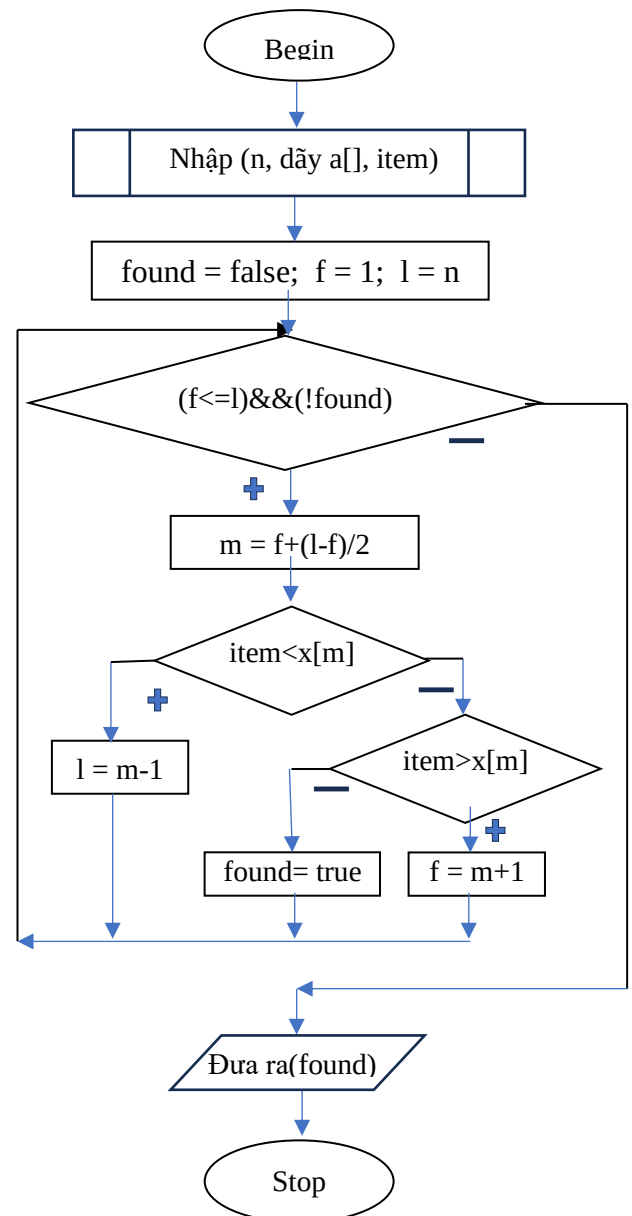
Với $n=10$ thì $2^{10}=1024$, nghĩa là ta đã xét được trong tập Δ có hơn một nghìn phần tử.

13.2. Vẽ sơ đồ khối thuật toán tìm phần tử trong một tập đã được sắp theo thuật toán tìm kiếm theo chia đôi (tìm kiếm nhị phân)

Tìm phần tử trong danh sách $x[1], x[2], \dots, x[n]$.
Đặt biến *found* sẽ có giá trị đúng TRUE khi tìm được và *mid* (middle – điểm giữa) là vị trí điểm chia đôi các phần để tìm, *item* – phần tử cần tìm.

1. *found* = FALSE; *first* = 1; *last* = *n* ; // *first* – điểm đầu, *last* – điểm cuối

```
2. while (( first <= last ) && (! found ))
    { mid = first + ( last - first ) / 2;
      if ( item < x[mid] ) last = mid - 1 ;
      else
      { if ( item > x[mid] ) first = mid + 1 ;
        else found = TRUE
      }
    }
```



13.3. Viết hàm C++ cho sơ đồ thuật toán đã vẽ ở mục 13.2

```

#include <iostream>
using namespace std;
int main()
{   cout << "\n TIM KIEM THEO CHI DOI " << endl;
    int i,n,item,first,last,a[20],found=0;
    cout << "\n Nhap n: ";cin>>n;
    for(i=1;i<=n;i++){cout << "\n Nhap a["<<i<<"]="<<";cin>>a[i]; }
    cout << "\n Nhap so can tim : ";cin>>item;
    first=1;last=n;
    while((first<last)&&!found){
        int mid=first+(last-first)/2;
        if(a[mid]==item){found=1;cout<<" Da tim thay item ="<<item<<" tai vi tri "<<mid; break;}
        if(a[mid]<item){first=mid+1;}
        else {last=mid-1;}
    }
    if(found==0){cout<<" Khong tim thay !";}
    return 0;
}

```

Câu 14.

14.1. Vẽ sơ đồ khối thuật toán Euclid tìm ước số chung lớn nhất (USCLN) của 2 số nguyên a,b.

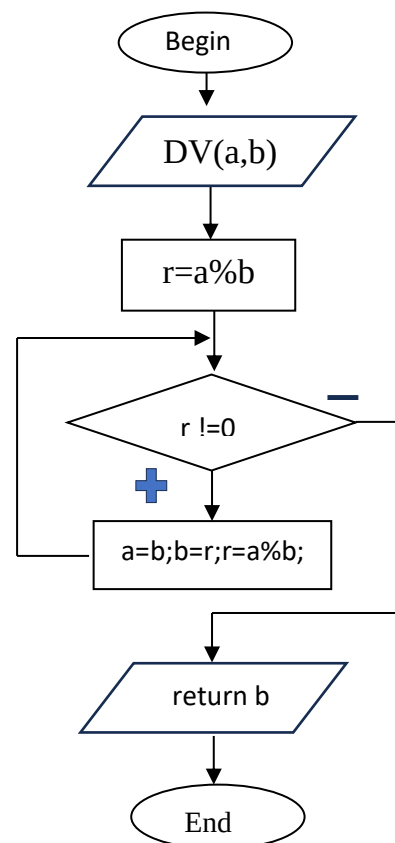
14.2. Viết hàm C++ cho sơ đồ khối thuật toán Euclid tìm ước số chung lớn nhất (USCLN) của 2 số nguyên a,b đã vẽ ở mục 14.1.

Thuật toán Euclid - tìm ước số chung lớn nhất

```

#include <iostream>
using namespace std;
int uscln(int m, int n)
{   int r ;   r = m % n;
    while ( r != 0){m = n; n = r; r = m % n ; }
    return n; }
int main()
{   cout << "\n\n Thuật toán Euclid tìm USCLN : " << endl;
    int a,b;
    cout<< "\n Nhap a,b " ; cin>>a>>b;
    cout<<" \n USCLN cua "<<a<<" va "<<b<<" la : ";
    cout << uscln(a,b)<<" \n\n";
    system("pause");
    return 0;
}

```



14.3. Hãy chứng minh rằng độ phức tạp của thuật toán tìm USCLN của Euclid có độ phức tạp : $O(\log N)$

Xét thời gian thực hiện thuật toán trong thân hàm `uscln(int m, int n)`

while (r != 0){m = n; n = r; r = m % n ; }

$$m = n \cdot q_1 + r_1, \quad 0 \leq r_1 < n$$

$$n = r_1 \cdot q_2 + r_2, \quad 0 \leq r_2 < r_1$$

Ta cần chứng minh mệnh đề sau đây : sau 2 lần lặp thì $r_2 < n/2$

Nhận định 1 : nếu $r_1 > n/2$ thì $r_2 < n/2$

Chứng minh nhận định 1. Nếu $r_1 > n/2$ thì suy ra $q_2 = 1$, vì nếu $q_2 \neq 1$ thì q_2 phải ≥ 2 , mà thế thì $(r_1 > n/2) \cdot (q_2 \geq 2) + r_2 > n$, là không thể được vì $n = r_1 \cdot q_2 + r_2$. Suy tiếp $\Rightarrow$ khi $q_2 = 1$ thì $n = r_1 \cdot q_2 + r_2 = r_1 + r_2$, mà $r_1 > n/2$ nên suy ra $\Rightarrow r_2 < n/2$.

Nhận định 2 : nếu $r_1 < n/2$ thì cũng suy ra $r_2 < n/2$

Chứng minh nhận định 2 : $r_2 = n \% r_1$, r_2 là phần dư của phép chia $n \% r_1$ nên $r_2 < r_1$ (số chia nhỏ hơn số bị chia), suy ra $r_2 < n/2$.

Từ Nhận định 1 và 2, suy ra trong mọi trường hợp, $r_2 < n/2$. ($4 < n/2 = 6$)

Ví dụ : $m=20, n=12; \quad 20=12 \cdot 1 + 8;$

$$12 = 8 \cdot 1 + 4;$$

Lần thực hiện thứ nhất : $m=20; n=12, r_1 = m \% n = 8$; vì $(r_1 \neq 0)$ nên lặp while

Lần thực hiện thứ hai : $m=12; n=8, r_2 = m \% n = 4$; vì $(r_2 \neq 0)$ nên while

....

Như vậy, cứ sau hai lần thực hiện thân vòng lặp thì số dư sẽ nhỏ hơn một nửa của n : $r < n/2$

Ta có thể quy nạp : 1 cặp lần lặp số dư sẽ $< n/2^1$

2 cặp lần lặp số dư sẽ $< n/2^2$

i cặp lần lặp số dư sẽ $< n/2^i$

Giả sử k là số nguyên lớn nhất sao cho $2^k \leq n$ để kết thúc thuật toán, số dư cuối cùng $= 0$, từ bất đẳng thức $2^k \leq n$ ta có $k \leq \log_2 n$ (xem lại cách tính logarit của 1 số : $n=2^k \Rightarrow k = \log_2 n$)

Kết quả, Độ phức tạp thuật toán sẽ là $O(\log_2 n)$.